Project 6: Crazyflie Fault-Tolerant Control

Slide guide · Team LUGER · what each slide says and what to say

The whole idea in 3 lines

- One motor at 50% RPM gives only 25% thrust (thrust $\propto$ RPM²).
- A Crazyflie has too little spare thrust (2.2–3.5:1, and 4:1 is needed), so the weak motor can't hold its corner.
- So we give up heading (yaw): the drone spins but stays level and on the path. MPC runs on the laptop; INDI and WLS run on the drone.

Slide 1 · Title · Shashank

"We're Team LUGER, Project 6. We're making a Crazyflie keep flying its path when one motor drops to half speed. The orange motor is the weak one."

Slide 2 · Objectives · Shashank

1. Fly a rectangle with all motors healthy (the baseline).
2. One motor is cut to 50% RPM and we're told which one.
3. The controller adapts.
4. Then finish the rectangle or land safely.
5. Log everything.

The demo is outdoors on the football ground. Detecting the fault is out of scope, because the brief says we're told.

Slide 3 · Why it's hard · Pavaneswar

- Half RPM gives a quarter of the thrust.
- To stay level *and* keep heading, each motor must carry ¼ of the weight. That needs thrust-to-weight of 4:1.
- A Crazyflie only has about 2.2:1 (standard) to 3.5:1 (Brushless).
- So the weak motor can't hold its corner, and we give up heading control. This is the standard approach in research (Mueller & D'Andrea, Sun et al.).

Slide 4 · Key idea · Pavaneswar

Standard Crazyflie, 27 g, which needs 6.75 g per motor to hover:

- The weak motor gives its max, 3.75 g.
- Its diagonal partner matches it at 3.75 g, so the drone stays level.
- The other two carry 9.75 g each, within their 15 g limit. The total is 27 g.
- Diagonal motors spin the same way, so the twists no longer cancel and the drone spins. We control only where the thrust points.

Slide 5 · Architecture · Jithendra

- Laptop: mission (the rectangle), position MPC, health check, logging.
- Drone, 500 times a second: INDI attitude control and WLS motor allocation.
- They talk over the Crazyflie radio. Orange boxes are our code.
- Why not everything on the laptop? The radio delay (~10 ms) is too slow to stabilise a spinning drone, so the fast loop stays onboard.

Slide 6 · Three control layers · Jithendra

- **MPC** (laptop): plans about a second ahead and never plans a move the damaged drone can't make. It also estimates wind and leans into it.
- **INDI** (drone): measures how the rotation is actually changing and corrects the difference. The weak motor and wind gusts get cancelled automatically, without an exact model.
- **WLS allocation** (drone): splits the work across the 4 motors, each within its own limit. Priority: level, then lift, then heading. Needed because the stock Crazyflie mixer assumes all motors are equally strong.

Slide 7 · When the fault message arrives · Vineel

The evaluator caps motor k, the laptop sends "motor k at 50%" over the radio, and the controller switches mode, ramping over ~0.2 s. Four changes:

- Motor k's limit drops to 25% of max.
- Heading is released.
- Allocation stops spending effort on heading.
- MPC limits tighten: ~2.5 motors' worth of thrust instead of 4, lower tilt and speed.

In our tests, one laptop command applies the cap and sends the message together.

Slide 8 · Demo mission · Vineel

- Flight 1: baseline A→B→C→D→A, all motors healthy.
- Flight 2: same path, with the fault applied on B→C.
- The drone settles for 2–3 s, then runs a health check (tilt, position, battery, wind).
- Pass: finish at half speed and land at A. Fail: land where it is. Both meet the brief.

Slide 9 · Outdoors + extreme cases · Hemanth

- No GPS on the Crazyflie, and Lighthouse fails in sunlight, so we use Loco UWB anchors (Flow deck as backup).
- Wind: INDI cancels gusts and MPC handles steady wind. We fly in calm hours, low, over grass.
- Extreme case (still in progress), a sudden drop after a lap: detect fast height loss, level the drone and use full thrust. If it's too low or upside down, cut the motors (the firmware's tumble check). The laptop kill switch is always available.

Slide 10 · Build and test plan · Hemanth

1. Python sim (PyBullet)
2. Firmware sim (CrazySim)
3. Baseline flights
4. Low fault tests over grass
5. Full demo

Fallback levels: A = spin and land safely (already meets the brief), B = hold position, C = finish the rectangle (the goal).

Slide 11 · Deliverables · Shaik Babu

D1–D8 follow the brief one to one, and D9 is logs and the report. Read the right column: it says how we prove each one. **Don't promise anything extra.** Deliverables are locked after today.

Slide 12 · Stretch goals and scope · Shaik Babu

- Stretch, not committed: finish the rectangle in every trial, recover from a sudden drop, test stock vs ours in real flight (we commit to that in simulation only).
- Out of scope: detecting the fault, a fully stopped motor, sensor failures.
- Our commitment: finish the rectangle *or* land safely.

Slide 13 · Components · Prasad

- Hardware: Crazyflie (from the lab), Loco deck and anchors, Crazyradio dongle, laptop, spare batteries and props.
- Software: Crazyflie firmware plus our app, cflib (Python), CasADi/acados for MPC, the Paparazzi WLS solver, PyBullet/CrazySim.
- All open source, and nothing to buy except spares.

Slide 14 · Summary · Prasad

Problem → approach → demo, in one line each. Then "Thank you, questions welcome."

Quick answers if asked

Why does it spin? The weak motor's diagonal partner has to drop to keep the drone level. Diagonal motors spin the same way, so the twists no longer cancel.

Isn't spinning out of control? No. Position and tilt stay controlled; only heading is free. This is the standard result in research.

Why INDI? It corrects based on what it measures, not on a model, so the fault and the wind are cancelled automatically.

Why MPC? After the fault, the problem is limits, and MPC plans only moves within them.

Why not run everything on the laptop? The radio delay is too slow for the 500 Hz stabilisation loop.

How do you know it's 50% RPM? 50% RPM = 25% thrust, calibrated on a thrust stand.

What if it can't hold position? The health check fails and it lands. That still meets the brief.

Will you detect the fault? No. The brief says we're told.